

oh-my-opencode 项目架构总览

生成时间：2026-02-27 | 基于 dev 分支分析

项目定位

oh-my-opencode 是一个 **OpenCode (Claude Code fork)** 插件，通过以下核心能力扩展 AI 编码体验：

- **11 个专业 Agent** 的多 Agent 编排系统
- **46 个生命周期 Hook** 覆盖安全/质量/稳定性/延续
- **26 个工具** 涵盖搜索/编辑/编排/LSP/Shell
- **Skill/命令/MCP** 三层扩展体系
- **Claude Code** 完全兼容

规模

指标	数值
TypeScript 文件	1208
代码行数	~143k LOC
Agent 数量	11
Hook 数量	46
工具数量	26
Feature 模块	19
配置 Schema 文件	22+

核心设计哲学

"正确的模型做正确的事" — 通过 Category 系统将任务域映射到最适合的 AI 模型：

模型	擅长领域
Claude Opus 4-6	编排、规划、高强度通用任务
GPT-5.3-Codex	深度逻辑推理、自主问题解决
Gemini 3.1 Pro	前端 UI/UX、高创意任务
Claude Haiku 4-5	快速小任务
Kimi K2.5	写作/文档
Grok Code Fast / MiniMax	低成本搜索任务

目录结构

```
oh-my-opencode/
├─ src/
│   │   └─ index.ts                # 插件入口
│   │   └─ plugin-interface.ts     # 8 个 OpenCode Hook Handler
│   │   └─ plugin-config.ts        # JSONC 多级配置加载
│   │   └─ plugin-state.ts         # 模型缓存状态
│   │   └─ create-hooks.ts         # Hook 三层组装
│   │   └─ create-managers.ts      # 4 大管理器创建
│   │   └─ create-tools.ts         # 工具体系创建
│   │
│   └─ agents/                    # 11 个 Agent 定义
│       │   └─ sisyphus.ts         # 主编排器
│       │   └─ hephaestus.ts      # 自主深度工作者
│       │   └─ atlas/              # Todo 编排执行器
│       │   └─ prometheus/         # 计划生成器 (8 个文件)
│       │   └─ metis.ts            # 计划前分析师
│       │   └─ momus.ts            # 计划审查员
│       │   └─ oracle.ts           # 战略顾问 (只读)
│       │   └─ explore.ts          # 代码库搜索 (只读)
│       │   └─ librarian.ts        # 外部知识搜索 (只读)
│       │   └─ multimodal-looker.ts # 多模态查看器
│       └─ dynamic-agent-prompt-builder.ts # 动态 Prompt 构建器
│
│   └─ tools/                     # 26 个工具
│       │   └─ delegate-task/      # 核心委派工具 task() (40+ 文件)
│       │   └─ task/               # 任务 CRUD 系统
│       │   └─ background-task/    # 后台任务管理
│       │   └─ ast-grep/           # AST 搜索
│       │   └─ grep/ & glob/       # 文本/文件搜索
│       │   └─ lsp/                # LSP 工具集
│       │   └─ session-manager/    # 会话管理
│       │   └─ skill/ & skill-mcp/ # 技能系统
│       │   └─ hashline-edit/      # Hashline 编辑
│       │   └─ look-at/            # 多模态查看
│       └─ interactive-bash/       # 交互式终端
│
│   └─ hooks/                     # 46 个 Hook 实现 (39 目录 + 6 文件)
│   └─ features/                  # 19 个功能模块
│   └─ config/                    # Zod v4 Schema 系统 (22+ 文件)
│   └─ plugin/                    # OpenCode Hook 处理器 + Hook 组合
│   └─ plugin-handlers/           # 6 阶段配置加载管线
│   └─ shared/                    # 100+ 工具函数 (13 类)
│   └─ mcp/                      # 3 个内置远程 MCP
│   └─ cli/                       # CLI: install, run, doctor, mcp-oauth
│
├─ packages/                     # 10 个平台二进制包
├─ assets/                       # JSON Schema
└─ docs/                         # 用户文档
```


初始化流程

插件入口 (src/index.ts)

```
// src/index.ts - 插件入口, 完整源码
const OhMyOpenCodePlugin: Plugin = async (ctx) => {
  log("[OhMyOpenCodePlugin] ENTRY - plugin loading", {
    directory: ctx.directory,
  })

  injectServerAuthIntoClient(ctx.client)    // 1. 注入服务端认证
  startTmuxCheck()                          // 2. Tmux 环境检测

  const pluginConfig = loadPluginConfig(ctx.directory, ctx) // 3. JSONC 多级配置
  const disabledHooks = new Set(pluginConfig.disabled_hooks ?? [])
  const isHookEnabled = (hookName: HookName): boolean => !disabledHooks.has(hookName)
  const safeHookEnabled = pluginConfig.experimental?.safe_hook_creation ?? true
  const firstMessageVariantGate = createFirstMessageVariantGate()

  const tmuxConfig = {
    enabled: pluginConfig.tmux?.enabled ?? false,
    layout: pluginConfig.tmux?.layout ?? "main-vertical",
    main_pane_size: pluginConfig.tmux?.main_pane_size ?? 60,
    main_pane_min_width: pluginConfig.tmux?.main_pane_min_width ?? 120,
    agent_pane_min_width: pluginConfig.tmux?.agent_pane_min_width ?? 40,
  }

  const modelCacheState = createModelCacheState()

  // 4. 创建 4 大管理器
  const managers = createManagers({
    ctx, pluginConfig, tmuxConfig, modelCacheState,
    backgroundNotificationHookEnabled: isHookEnabled("background-notification"),
  })

  // 5. 创建工具体系 (26 个工具)
  const toolsResult = await createTools({ ctx, pluginConfig, managers })

  // 6. 创建 46 个 Hook (三层架构)
  const hooks = createHooks({
    ctx, pluginConfig, modelCacheState,
    backgroundManager: managers.backgroundManager,
    isHookEnabled, safeHookEnabled,
    mergedSkills: toolsResult.mergedSkills,
    availableSkills: toolsResult.availableSkills,
  })

  // 7. 组装 OpenCode 插件接口
  const pluginInterface = createPluginInterface({
    ctx, pluginConfig, firstMessageVariantGate,
    managers, hooks, tools: toolsResult.filteredTools,
```

```

})

return {
  ...pluginInterface,
  // 实验性 Hook: 会话压缩
  "experimental.session.compacting": async (
    _input: { sessionId: string },
    output: { context: string[] },
  ): Promise<void> => {
    await hooks.compactionTodoPreserver?.capture(_input.sessionID)
    await hooks.claudeCodeHooks?.["experimental.session.compacting"]?._input, output)
    if (hooks.compactionContextInjector) {
      output.context.push(hooks.compactionContextInjector(_input.sessionID))
    }
  },
}
}

export default OhMyOpenCodePlugin

```

管理器创建 (src/create-managers.ts)

```
// src/create-managers.ts - 4 大管理器工厂
export type Managers = {
  tmuxSessionManager: TmuxSessionManager
  backgroundManager: BackgroundManager
  skillMcpManager: SkillMcpManager
  configHandler: ReturnType<typeof createConfigHandler>
}

export function createManagers(args: {
  ctx: PluginContext
  pluginConfig: OhMyOpenCodeConfig
  tmuxConfig: TmuxConfig
  modelCacheState: ModelCacheState
  backgroundNotificationHookEnabled: boolean
}): Managers {
  const { ctx, pluginConfig, tmuxConfig, modelCacheState, backgroundNotificationHookEnabled } = args

  const tmuxSessionManager = new TmuxSessionManager(ctx, tmuxConfig)
  const backgroundManager = new BackgroundManager(
    ctx,
    pluginConfig.background_task,
    {
      tmuxConfig,
      onSubagentSessionCreated: async (event: SubagentSessionCreatedEvent) => {
        await tmuxSessionManager.onSessionCreated({
          type: "session.created",
          properties: {
            info: { id: event.sessionID, parentID: event.parentID, title: event.title },
          },
        })
      },
      onShutdown: () => {
        tmuxSessionManager.cleanup().catch((error) => {
          log("[index] tmux cleanup error during shutdown:", error)
        })
      },
      enableParentSessionNotifications: backgroundNotificationHookEnabled,
    },
  )

  initTaskToastManager(ctx.client)
  const skillMcpManager = new SkillMcpManager()
  const configHandler = createConfigHandler({
    ctx: { directory: ctx.directory, client: ctx.client },
    pluginConfig, modelCacheState,
  })

  return { tmuxSessionManager, backgroundManager, skillMcpManager, configHandler }
}
```

工具创建 (src/create-tools.ts)

```
// src/create-tools.ts - 工具体系工厂
export async function createTools(args: {
  ctx: PluginContext
  pluginConfig: OhMyOpenCodeConfig
  managers: Pick<Managers, "backgroundManager" | "tmuxSessionManager" | "skillMcpManager">
}): Promise<CreateToolsResult> {
  const { ctx, pluginConfig, managers } = args

  const skillContext = await createSkillContext({ directory: ctx.directory, pluginConfig })
  const availableCategories = createAvailableCategories(pluginConfig)

  const { filteredTools, taskSystemEnabled } = createToolRegistry({
    ctx, pluginConfig, managers, skillContext, availableCategories,
  })

  return {
    filteredTools,
    mergedSkills: skillContext.mergedSkills,
    availableSkills: skillContext.availableSkills,
    availableCategories,
    browserProvider: skillContext.browserProvider,
    disabledSkills: skillContext.disabledSkills,
    taskSystemEnabled,
  }
}
```


Hook 三层组装 (src/create-hooks.ts)

```
// src/create-hooks.ts — 46 个 Hook 的三层组装
export function createHooks(args: {
  ctx: PluginContext
  pluginConfig: OhMyOpenCodeConfig
  modelCacheState: ModelCacheState
  backgroundManager: BackgroundManager
  isHookEnabled: (hookName: HookName) => boolean
  safeHookEnabled: boolean
  mergedSkills: LoadedSkill[]
  availableSkills: AvailableSkill[]
}) {
  const { ctx, pluginConfig, modelCacheState, backgroundManager,
    isHookEnabled, safeHookEnabled, mergedSkills, availableSkills } = args

  // 第一层: 核心 Hook (37 个) — 会话 + 工具守卫 + 变换
  const core = createCoreHooks({
    ctx, pluginConfig, modelCacheState, isHookEnabled, safeHookEnabled,
  })

  // 第二层: 延续 Hook (7 个) — 延续/压缩/通知
  const continuation = createContinuationHooks({
    ctx, pluginConfig, isHookEnabled, safeHookEnabled,
    backgroundManager, sessionRecovery: core.sessionRecovery,
  })

  // 第三层: 技能 Hook (2 个) — 技能提醒/自动命令
  const skill = createSkillHooks({
    ctx, isHookEnabled, safeHookEnabled, mergedSkills, availableSkills,
  })

  return { ...core, ...continuation, ...skill }
}
```

8 个 OpenCode Hook Handler

src/plugin-interface.ts — createPluginInterface() 将所有子系统组装为 OpenCode 期望的 Hook 接口:

```
// src/plugin-interface.ts - 完整源码
export function createPluginInterface(args: {
  ctx: PluginContext
  pluginConfig: OhMyOpenCodeConfig
  firstMessageVariantGate: { /* ... */ }
  managers: Managers
  hooks: CreatedHooks
  tools: ToolsRecord
}): PluginInterface {
  const { ctx, pluginConfig, firstMessageVariantGate, managers, hooks, tools } = args

  return {
    tool: tools, // 26 个工具

    "chat.params": async (input, output) => { // Anthropic effort 调整
      const handler = createChatParamsHandler({ anthropicEffort: hooks.anthropicEffort })
      await handler(input, output)
    },

    "chat.headers": createChatHeadersHandler({ ctx }), // 请求头注入

    "chat.message": createChatMessageHandler({ // 消息处理
      ctx, pluginConfig, firstMessageVariantGate, hooks,
    }),

    "experimental.chat.messages.transform": createMessagesTransformHandler({ hooks }), // 上下文注入

    "experimental.chat.system.transform": createSystemTransformHandler(), // 系统 prompt

    config: managers.configHandler, // 6 阶段配置管线

    event: createEventHandler({ // 会话生命周期事件
      ctx, pluginConfig, firstMessageVariantGate, managers, hooks,
    }),

    "tool.execute.before": createToolExecuteBeforeHandler({ ctx, hooks }), // 工具前置守卫

    "tool.execute.after": createToolExecuteAfterHandler({ hooks }), // 工具后置处理
  }
}
```

处理器	源文件	功能
config	plugin-handlers/config-handler.ts	6 阶段配置加载管线
tool	plugin/tool-registry.ts	26 个工具注册
chat.message	plugin/chat-message.ts	首次消息变体、会话设置、关键词检测
chat.params	plugin/chat-params.ts	Anthropic effort 调整

处理器	源文件	功能
event	plugin/event.ts	会话生命周期 (created/deleted/idle/error)
tool.execute.before	plugin/tool-execute-before.ts	文件守卫、标签截断、 规则注入
tool.execute.after	plugin/tool-execute-after.ts	输出截断、元数据存储
experimental.chat.messages.transform	plugin/messages-transform.ts	上下文注入、思考块验证

关键入口文件

文件	行数	职责
src/index.ts	~110	插件入口，组装所有子系统
src/plugin-interface.ts	~80	创建 8 个 OpenCode Hook Handler
src/create-managers.ts	~90	4 大管理器工厂
src/create-tools.ts	~60	工具体系工厂
src/create-hooks.ts	~65	Hook 三层组装
src/plugin-config.ts	~200	JSONC 配置加载/合并/验证
src/plugin-state.ts	~15	模型缓存状态

文档目录

文档	内容
00-overview.md	项目架构总览（本文）
01-agents.md	11 个 Agent 详解
02-multi-agent-orchestration.md	多 Agent 编排原理
03-plan-build-mode.md	Plan/Build 模式端到端流程
04-tools-and-hooks.md	工具系统与 Hook 体系
05-config-features-mcp.md	配置系统、Feature 模块、MCP 体系